UNIVERSITÉ ALIOUNE DIOP DE BAMBEY

UFR Sciences & Technologies | Master 2 Big Data & Intelligence Artificielle

Année universitaire 2025-2026

PROJET TUTORÉ — CODES SOURCES COMPLETS

🐟 Cayar-SN

Intelligence Halieutique & Optimisation des Sorties en Mer

Pêche Artisanale & Ressources Halieutiques

Durée	4 semaines
Équipe	2 étudiants
Domaine	Pêche Artisanale & Ressources Halieutiques
Standard	M2 Approche Industrie | New Deal Technologique Sénégal
Enseignant	Mr Ahmed Ben Sidy Bouya SEYE — Senior Big Data & AI Engineer | Groupe Sonatel

NiFi • Kafka • Spark Streaming • Pandera • HBase • Hive • Airflow • MLlib • Privacy by Design

1. Contexte et Objectifs

La pêche artisanale représente 75% des captures nationales et fait vivre 600 000 personnes au Sénégal. La surpêche et la variabilité climatique menacent les stocks. Ce projet construit un système de recommandation pour optimiser les sorties en mer des pirogues : meilleurs créneaux, zones de pêche, et prédiction de la ressource disponible (CRODT).

> ℹ️ **New Deal Technologique Sénégal**
> Ce projet s'inscrit dans la Stratégie Sénégal Numérique 2025-2035 et le Plan Sénégal Émergent. L'objectif est de former des ingénieurs Big Data capables de construire des solutions Data-Driven directement applicables aux défis de développement du Sénégal.

1.1 Objectifs Techniques

#	Composant	Description
1	**Ingestion multi-sources**	données CRODT (stocks), météo marine ANACIM, GPS pirogues
2	**Prédiction des zones de pêche optimales par espèce (sardinelle, thiof, yabois)**	Prédiction des zones de pêche optimales par espèce (sardinelle, thiof, yabois)
3	**Alerte météo marine**	vent > 15 nœuds, houle > 2m (sécurité des pêcheurs)
4	**Privacy by Design**	anonymisation des identifiants de pirogues (données opérationnelles)
5	**Dashboard pêcheurs**	carte des zones recommandées, alertes sécurité, quotas respectés

1.2 Architecture End-to-End

📄 Architecture

```
[ SOURCES ]  ──▶  [ NiFi ]  ──▶  [ KAFKA ]  ──▶  [ SPARK ML ]  ──▶  [ STORAGE ]
 CRODT stocks     Orchestration   Topic:          Prédiction          HBase/HDFS
 Météo marine     & Routage       cayar_meteo     Zone optimale       Gold Layer
 GPS pirogues                     cayar_captures  Alerte sécurité          |
                                                        |            [ ANALYTICS ]
                                                        ▼              Apache Hive
                                              [ ALERT LAYER ]        vue_zones_pche
                                              Vent/Houle ROUGE       vue_especes
```

2. Prérequis & Infrastructure

Configuration minimale

RAM : 16 Go. CPU : 4 cœurs. OS : Ubuntu 22.04 LTS. Docker Engine 24.x. Java 11 requis par Spark.

setup.sh

```
# Démarrage infrastructure
mkdir -p data dags models nifi_templates
docker compose up -d zookeeper && sleep 10
docker compose up -d kafka nifi hbase hive-metastore && sleep 30
docker compose up -d spark-master spark-worker airflow

# Initialiser HBase (une seule fois)
python hbase_setup.py

# Démarrer le simulateur
python kafka_producer_cayar_sn.py &

# Interfaces web :
#   NiFi    : http://localhost:8081
#   Spark   : http://localhost:8080
#   HBase   : http://localhost:16010
#   Airflow : http://localhost:8082
```

requirements.txt

```
# requirements.txt — Cayar-SN
pyspark==3.3.2
kafka-python==2.0.2
pandera==0.17.2
pandas==2.0.3
apache-airflow==2.7.3
mlflow==2.8.1
pyhive==0.7.0
happybase==1.2.0
matplotlib==3.7.3
seaborn==0.12.2
faker==30.1.0
scipy==1.11.4
numpy==1.24.4
```

3. Codes Sources Complets & Fonctionnels

3.1 Initialisation HBase — hbase_setup.py

> ⚠️ **Fichier obligatoire**
> Crée les 3 tables HBase de Cayar-SN. À exécuter avant le démarrage du pipeline.

📄 **hbase_setup.py**

```
# hbase_setup.py — Tables HBase Cayar-SN
import happybase, logging
logger = logging.getLogger('HBaseSetup')

def create_cayar_sn_tables():
    conn = happybase.Connection('hbase', port=9090, timeout=10000)
    conn.open()
    tables = {
        b'cayar:conditions_mer': {'b'meta': {'max_versions': 1}},  # Conditions météo
marine en temps réel
        b'cayar:alertes_securite': {'b'meta': {'max_versions': 1}},  # Alertes vent/houle
par zone (TTL 12h)
        b'cayar:zones_recommandees': {'b'meta': {'max_versions': 1}},  # Carte zones de
pêche optimales
    }
    existantes = [t.decode() for t in conn.tables()]
    for nom_b, fam in tables.items():
        nom = nom_b.decode()
        if nom not in existantes:
            conn.create_table(nom_b, fam)
            logger.info(f'Table {nom} créée ✓')
    conn.close()

if __name__ == '__main__':
    logging.basicConfig(level=logging.INFO)
    create_cayar_sn_tables()
```

3.2 Simulateur de Données — kafka_producer

> ℹ️ **Données simulées réalistes**
> Le simulateur génère des données calées sur les réalités de Pêche Artisanale & Ressources Halieutiques au Sénégal. Les champs PII (noms, IDs) sont présents dans le simulateur et seront anonymisés par la Privacy Layer Spark.

📄 **kafka_producer_cayar_sn.py**

```
# kafka_producer_cayar.py — Simulateur météo marine + captures
from kafka import KafkaProducer
import json, random, time, uuid
from datetime import datetime
import numpy as np

random.seed(42); np.random.seed(42)
ZONES_MER   =
['CAYAR','SAINT_LOUIS','JOAL','MBOUR','DAKAR_PORT','KAFOUNTINE','ZIGUINCHOR']
ESPECES     = ['SARDINELLE','THIOF','YABOIS','CAPITAINE','SOLE','CREVETTE']
# Coordonnées zones de pêche (approximatif)
GPS_ZONES = {
    'CAYAR':        (14.92, -17.16), 'SAINT_LOUIS':(16.01,-16.50),
    'JOAL':         (14.16, -16.85), 'MBOUR':      (14.40,-16.96),
    'DAKAR_PORT':   (14.67, -17.43), 'KAFOUNTINE':(12.86,-16.73),
    'ZIGUINCHOR':   (12.55, -16.27),
```

```
}
# Abondance par espèce et saison (nov-mars = saison froide, meilleure pêche)
ABONDANCE_SAISON = {
    'SARDINELLE': lambda m: 1.5 if m in [11,12,1,2,3] else 0.7,
    'THIOF':      lambda m: 1.2 if m in [10,11,12,1,2] else 0.8,
    'YABOIS':     lambda m: 1.0,
    'CAPITAINE':  lambda m: 1.3 if m in [6,7,8,9] else 0.6,
    'SOLE':       lambda m: 1.1, 'CREVETTE': lambda m: 1.4 if m in [3,4,5] else 0.5,
}

producer = KafkaProducer(bootstrap_servers=['kafka:9092'],
    value_serializer=lambda v: json.dumps(v).encode())

def gen_meteo_marine():
    zone  = random.choice(ZONES_MER)
    m     = datetime.utcnow().month
    # Harmattan (nov-mars) : vents forts
    vent_base = 12 if m in [11,12,1,2,3] else 8
    return {
        'zone':           zone,
        'vent_noeuds':    round(max(0, np.random.normal(vent_base, 4)), 1),
        'houle_metres':   round(max(0, np.random.normal(1.2, 0.6)), 2),
        'temp_eau_celsius': round(np.random.normal(24, 3), 1),
        'visibilite_km':  round(random.uniform(2, 30), 1),
        'timestamp':      datetime.utcnow().strftime('%Y-%m-%dT%H:%M:%SZ'),
    }

def gen_capture():
    zone    = random.choice(ZONES_MER)
    espece  = random.choice(ESPECES)
    m       = datetime.utcnow().month
    factor  = ABONDANCE_SAISON[espece](m)
    return {
        'pirogue_id':   f'PIR_{uuid.uuid4().hex[:8].upper()}',  # Sera anonymisé
        'nom_patron':   f'Patron_{uuid.uuid4().hex[:4]}',        # Sera supprimé
        'zone':         zone,
        'espece':       espece,
        'capture_kg':   round(max(0, np.random.normal(250*factor, 80)), 1),
        'duree_mer_h':  round(random.uniform(4, 16), 1),
        'latitude':     round(GPS_ZONES[zone][0] + np.random.normal(0,0.2), 4),
        'longitude':    round(GPS_ZONES[zone][1] + np.random.normal(0,0.2), 4),
        'timestamp':    datetime.utcnow().strftime('%Y-%m-%dT%H:%M:%SZ'),
    }

if __name__ == '__main__':
    print('Simulateur Cayar-SN démarré...')
    while True:
        for zone in ZONES_MER: producer.send('cayar_meteo_raw', gen_meteo_marine())
        producer.send('cayar_captures_raw', gen_capture())
        producer.flush(); time.sleep(5)
```

3.3 Pipeline Spark Streaming — streaming

> ⚠️ **Privacy critique**
> Drop des champs PII après anonymisation SHA-256. Les données sensibles ne doivent jamais apparaître dans HBase ou Hive.

streaming_cayar_sn.py

```
# streaming_cayar.py — Pipeline Spark Streaming Cayar-SN
# Alertes sécurité maritime + zones de pêche optimales
import os
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, sha2, concat, lit, from_json, to_json, struct,
    window, avg, sum as spark_sum, when, coalesce, current_timestamp
)
from pyspark.sql.types import *

SALT = os.environ.get('CAYAR_SECRET_SALT','default_salt')
spark = SparkSession.builder.appName('Cayar_SN_Streaming') \
    .config('spark.sql.shuffle.partitions','4').getOrCreate()
spark.sparkContext.setLogLevel('WARN')

meteo_schema = StructType([
    StructField('zone',              StringType(), True),
    StructField('vent_noeuds',       FloatType(),  True),
    StructField('houle_metres',      FloatType(),  True),
    StructField('temp_eau_celsius', FloatType(),  True),
    StructField('visibilite_km',     FloatType(),  True),
    StructField('timestamp',         StringType(), True),
])

capt_schema = StructType([
    StructField('pirogue_id',  StringType(), True),
    StructField('nom_patron',  StringType(), True),
    StructField('zone',        StringType(), True),
    StructField('espece',      StringType(), True),
    StructField('capture_kg',  FloatType(),  True),
    StructField('duree_mer_h', FloatType(),  True),
    StructField('latitude',    FloatType(),  True),
    StructField('longitude',   FloatType(),  True),
    StructField('timestamp',   StringType(), True),
])

meteo_df = (spark.readStream.format('kafka')
    .option('kafka.bootstrap.servers','kafka:9092')
    .option('subscribe','cayar_meteo_raw').load()
    .select(from_json(col('value').cast('string'), meteo_schema).alias('d')).select('d.*')
    .withColumn('event_ts', current_timestamp())
    .withColumn('alerte_securite',
        when((coalesce('vent_noeuds',lit(0)) > 25) | (coalesce('houle_metres',lit(0)) >
3), lit('ROUGE'))
        .when((coalesce('vent_noeuds',lit(0)) > 15) | (coalesce('houle_metres',lit(0)) >
2), lit('ORANGE'))
        .otherwise(lit('VERT')))
)

capt_df = (spark.readStream.format('kafka')
    .option('kafka.bootstrap.servers','kafka:9092')
    .option('subscribe','cayar_captures_raw').load()
    .select(from_json(col('value').cast('string'), capt_schema).alias('d')).select('d.*')
    .withColumn('event_ts', current_timestamp())
    .withColumn('pirogue_secure', sha2(concat(col('pirogue_id'),lit(SALT)),256))
    .drop('pirogue_id','nom_patron')
)

# Agrégation zones de pêche (fenêtre 6h)
zones_opt = (capt_df
    .withWatermark('event_ts','1 hour')
    .groupBy(window('event_ts','6 hours','1 hour'), 'zone','espece')
```

```
    .agg(
        avg('capture_kg').alias('capture_moy_kg'),
        spark_sum('capture_kg').alias('capture_totale_kg'),
        avg('duree_mer_h').alias('duree_moy_h'),
    )
    .withColumn('rendement_kg_h',
        coalesce(col('capture_moy_kg'),lit(0)) / (coalesce(col('duree_moy_h'),lit(1))))
)

q1 = (meteo_df.select(to_json(struct('*')).alias('value'))
    .writeStream.format('kafka')
    .option('kafka.bootstrap.servers','kafka:9092')
    .option('topic','cayar_alerts')
    .option('checkpointLocation','/tmp/cayar_meteo_ckpt')
    .outputMode('append').start())
q2 = (zones_opt.select(to_json(struct('*')).alias('value'))
    .writeStream.format('kafka')
    .option('kafka.bootstrap.servers','kafka:9092')
    .option('topic','cayar_zones_agg')
    .option('checkpointLocation','/tmp/cayar_zones_ckpt')
    .outputMode('update').start())
q1.awaitTermination()
```

3.4 Tables Hive & Vues Analytics — hive_setup.sql

Les vues Hive utilisent systématiquement COALESCE sur les colonnes nullables pour éviter les NULL silencieux dans les agrégations.

hive_setup.sql

```
-- hive_setup.sql — Tables Hive Cayar-SN
CREATE DATABASE IF NOT EXISTS cayar_sn COMMENT 'Pêche Artisanale Sénégal — UADB 2025';
USE cayar_sn;

CREATE TABLE IF NOT EXISTS captures_historique (
    pirogue_secure STRING, zone STRING, espece STRING,
    capture_kg DOUBLE, duree_mer_h DOUBLE,
    latitude DOUBLE, longitude DOUBLE,
    ingestion_ts TIMESTAMP
) PARTITIONED BY (date_obs STRING)
STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');

CREATE TABLE IF NOT EXISTS conditions_mer_historique (
    zone STRING, vent_noeuds DOUBLE, houle_metres DOUBLE,
    temp_eau_celsius DOUBLE, alerte_securite STRING,
    ingestion_ts TIMESTAMP
) STORED AS ORC;

-- Vue : Zones de pêche optimales par espèce
CREATE OR REPLACE VIEW vue_zones_peche AS
SELECT zone, espece,
    AVG(capture_kg)                           AS capture_moy_kg,
    AVG(capture_kg / COALESCE(duree_mer_h,1))  AS rendement_kg_h,
    COUNT(pirogue_secure)                     AS nb_sorties
FROM captures_historique
WHERE date_obs >= DATE_SUB(CURRENT_DATE(),30)
GROUP BY zone, espece
ORDER BY rendement_kg_h DESC;

-- Vue : Sécurité maritime par zone
CREATE OR REPLACE VIEW vue_securite_maritime AS
SELECT zone,
    AVG(COALESCE(vent_noeuds,0))  AS vent_moy,
    AVG(COALESCE(houle_metres,0)) AS houle_moy,
    MAX(CASE WHEN alerte_securite='ROUGE' THEN 1 ELSE 0 END) AS alerte_rouge_active
FROM conditions_mer_historique
WHERE ingestion_ts >= DATE_SUB(CURRENT_TIMESTAMP, 6/24.0)
GROUP BY zone
ORDER BY vent_moy DESC;
```

3.5 DAG Airflow MLOps

Le BranchPythonOperator détecte automatiquement les dérives et déclenche le réentraînement du modèle si nécessaire. Les recommandations sont écrites dans HBase pour un accès temps réel.

dags/cayar_sn_dag.py

```
# dags/cayar_sn_dag.py — DAG Airflow Cayar-SN
from airflow import DAG
from airflow.operators.python import PythonOperator, BranchPythonOperator
from airflow.operators.dummy  import DummyOperator
from airflow.utils.dates      import days_ago
from datetime import timedelta
import subprocess, logging

logger = logging.getLogger('cayar_dag')
default_args = {'owner':'seye_ahmed','retries':2,'retry_delay':timedelta(minutes=5)}
```

```
def check_conditions(**ctx):
    from pyhive import hive
    conn = hive.Connection(host='hive-metastore',port=10000)
    cur  = conn.cursor()
    cur.execute("SELECT COUNT(*) FROM cayar_sn.vue_securite_maritime WHERE
alerte_rouge_active=1")
    n = cur.fetchone()[0]
    logger.info(f'Zones ROUGE actives : {n}')
    return 'retrain_model' if n >= 3 else 'update_recommendations'

def retrain_model(**ctx):
    subprocess.run(['spark-submit','--master','spark://spark-master:7077',
        '/opt/models/train_zone_optimale.py'],check=True,timeout=3600)

def update_recommendations(**ctx):
    import happybase; from pyhive import hive; from datetime import datetime
    conn_hbase = happybase.Connection('hbase',port=9090); conn_hbase.open()
    conn_hive  = hive.Connection(host='hive-metastore',port=10000)
    cur = conn_hive.cursor()
    cur.execute('SELECT zone,espece,capture_moy_kg,rendement_kg_h FROM
cayar_sn.vue_zones_peche LIMIT 50')
    table = conn_hbase.table(b'cayar:zones_recommandees')
    for row in cur.fetchall():
        zone,espece,cap,rend = row
        table.put(f'{zone}_{espece}'.encode(),{
            b'meta:zone':zone.encode(), b'meta:espece':espece.encode(),
            b'predict:capture_moy':str(round(cap,1)).encode(),
            b'predict:rendement':str(round(rend,2)).encode(),
            b'predict:ts':datetime.utcnow().isoformat().encode()})
    conn_hbase.close()

with DAG('cayar_sn_monitoring', default_args=default_args,
         schedule_interval='0 4 * * *', start_date=days_ago(1), catchup=False,
         tags=['cayar-sn','peche','securite-maritime']) as dag:
    start  = DummyOperator(task_id='start'); end = DummyOperator(task_id='end')
    branch = BranchPythonOperator(task_id='check_conditions',
                                  python_callable=check_conditions,provide_context=True)
    t_train=
PythonOperator(task_id='retrain_model',python_callable=retrain_model,provide_context=True)
    t_reco =
PythonOperator(task_id='update_recommendations',python_callable=update_recommendations,pro
vide_context=True)
    start >> branch >> [t_train, t_reco]
    t_train >> t_reco >> end; t_reco >> end
```

4. Livrables Attendus

#	Livrable	Contenu attendu
1	**Pipeline de Données**	Code source complet (GitHub) + template NiFi. Données visibles dans HDFS, topic Kafka cayar_meteo_raw actif.
2	**Modèle ML & MLOps**	Script d'entraînement + RMSE/R^2 documentés + DAG Airflow avec BranchPythonOperator fonctionnel.
3	**Catalogue Hive**	Tables créées, vues fonctionnelles avec au moins 1 alerte visible dans HBase en démonstration.
4	**Dashboard & Recommandations**	Dashboard graphique (Matplotlib) + recommandations concrètes contextualisées au Pêche Artisanale & Ressources Halieutiques sénégalais.

5. Barème de Notation — /20 points

> ℹ️ **Barème différencié 4 niveaux**
> Chaque critère est évalué selon 4 niveaux de maîtrise. La démonstration live en soutenance est obligatoire. Bonus documentation +2 pts.

Critère	Pts	Insuffisant (0)	Passable (50%)	Bien (75%)	Excellent (100%)
Ingestion NiFi Multi-Sources	3 pts	NiFi absent	1 source/3	CRODT + météo marine	3 sources (CRODT+météo+GPS), 2 topics distincts, messages géolocalisés
Validation & Privacy	4 pts	Aucune validation	Schéma partiel	SHA-256 sans drop pirogue_id	Pandera + drop(immat_pirogue, nom_patron) + 0 PII dans HBase
Streaming & Alertes Météo Marine	6 pts	Pas de Spark	Lecture seule	Alertes vent basiques	Classification 3 niveaux (VERT/ORANGE/ROUGE) + fenêtres + watermarks
Prédiction Zones & MLOps	4 pts	Aucun modèle	Règles fixes	RF entraîné sans DAG	RF zones optimales + DAG quotidien + F1/Recall documentés + MLflow
Dashboard & Recommandations Pêcheurs	3 pts	Aucun dashboard	KPIs sans carte	Carte zones + pas de quota	Carte zones optimales + alertes sécurité + recommandation quotas espèces
TOTAL	20 pts				

Bonus Documentation (jusqu'à +2 pts)

Élément	Points bonus
README.md avec docker compose up + captures d'écran	+0.5 pt
Commentaires de code en français + docstrings	+0.5 pt
Tests pytest sur les schémas Pandera et les UDFs	+0.5 pt
Démo live d'une alerte HBase sans erreur en soutenance	+0.5 pt

🐟 **Bon courage pour Cayar-SN !**

"Always be a solution, never a problem."

Mr Ahmed Ben Sidy Bouya SEYE | Senior Big Data & AI Engineer | Groupe Sonatel